

B.Sc. (Data Science)

DS101T : Problem Solving and Python Programming

Unit 3: Classes, Objects and Inheritance

Mrs. Reshma Masurekar
Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18

Classes, Objects and Inheritance

- **Classes and Objects**
- Classes as User Defined Data Type
- Objects as Instances of Classes
- Creating Class and Objects
- Creating Objects By Passing Values
- Variables & Methods in a Class

- **Inheritance**
- Single Inheritance
- Multilevel
- Multiple Inheritance
- Hybrid Inheritance
- Hierarchical Inheritance
- IS-A Relationship and HAS-A Relationship

Inheritance

- **Inheritance** is a fundamental concept in Object-Oriented Programming (OOP) that allows one class (the child or subclass) to inherit the attributes and methods of another class (the parent or superclass). This promotes code reuse and creates a hierarchical relationship between classes.

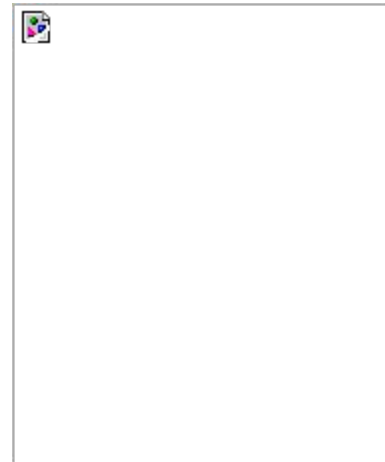
- **Syntax:**

```
Class BaseClass:
```

```
    #Body
```

```
Class DerivedClass(BaseClass):
```

```
    #Body
```



Key Concepts of Inheritance

1. **Parent Class (Superclass) and Child Class (Subclass):**

The parent class is the class being inherited from.

The child class inherits attributes and methods from the parent class and can also have additional attributes and methods.

2. **Overriding Methods:**

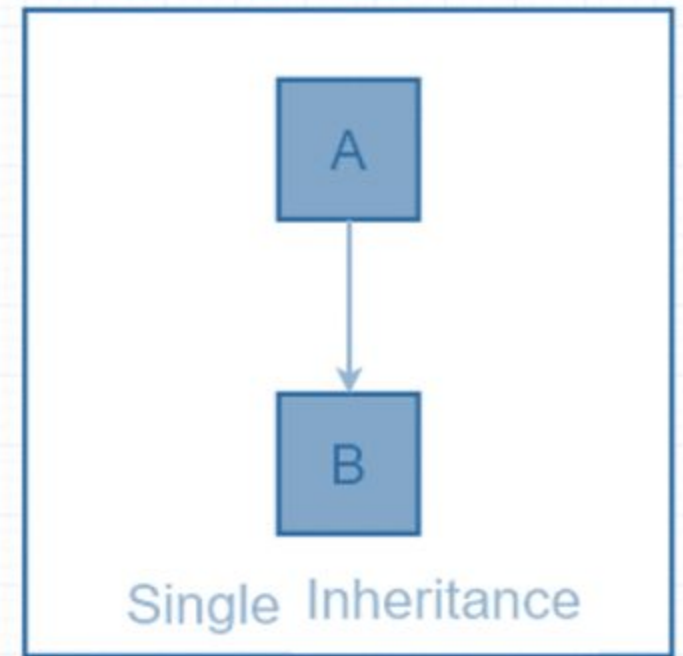
A child class can override methods of the parent class. This means the child class provides a specific implementation of a method that is already defined in the parent class.

3. **super() Function:**

The `super()` function is used to call a method from the parent class. This is particularly useful when you override a method in the child class but still want to retain some behavior from the parent class.

Single Inheritance

- Single inheritance enables a derived class to inherit properties from a single parent class, thus enabling code reusability and the addition of new features to existing code.
- Syntax:
 class classA:
 <class-suite>
 class classB(classA):
 <class suite>



```
class Parent:  
    def func_1(self):  
        print ("This function is defined inside the parent class.")
```

now, we will create the Derived class

```
class Child(Parent1):  
    def func_2(self):  
        print ("This function is defined inside the child class.")
```

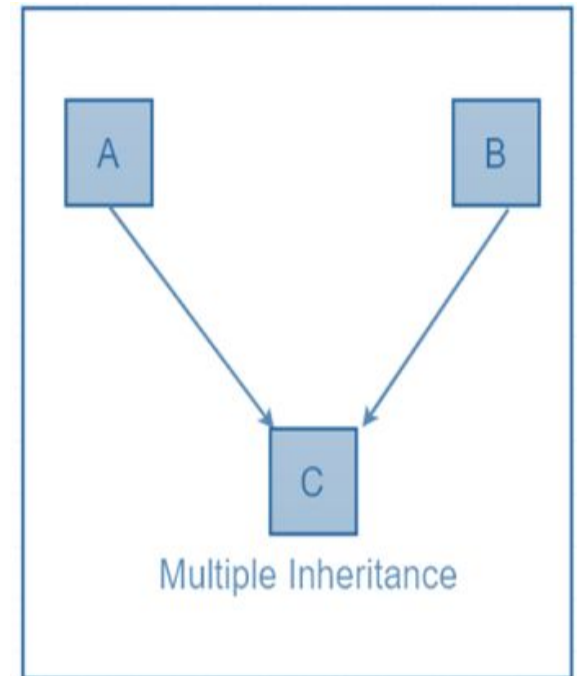
Driver's code

```
object = Child()  
object.func_1()  
object.func_2()
```

Multiple Inheritance

- When a class can be derived from more than one base class this type of inheritance is called multiple inheritances.
- In multiple inheritances, all the features of the base classes are inherited into the derived class.
- When there is multiple Inheritance, each of the attributes that are present in the classes of the base has been passed on to the class that is derived from it.
- **Syntax:**

```
class classA:  
    <class-suite>  
class classB(classA):  
    <class suite>  
class classC(classB):  
    <class suite>
```



```
class Mother1:
    mothername1 = ""
    def mother1(self):
        print(self.mothername1)

class Father1:
    fathername1 = ""
    def father1(self):
        print(self.fathername1)

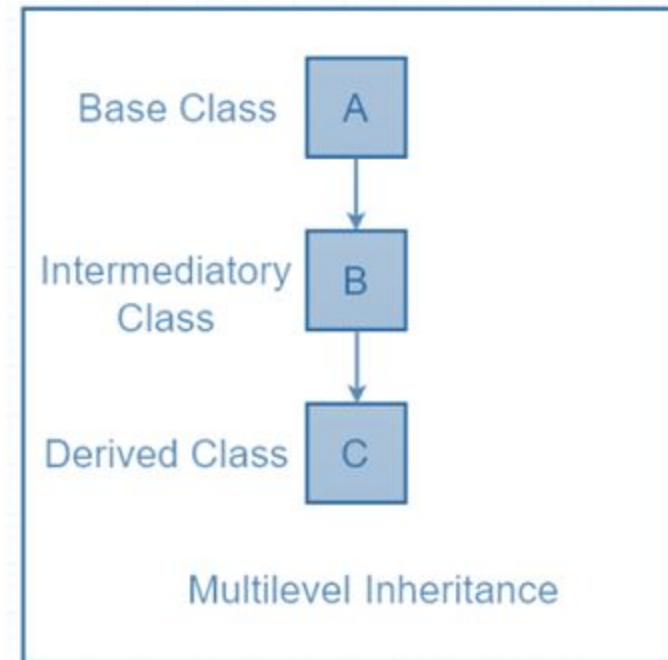
class Son1(Mother1, Father1):
    def parents1(self):
        print ("Father name is :", self.fathername1)
        print ("Mother name is :", self.mothername1)

s1 = Son1()
s1.fathername1 = "Rajesh"
s1.mothername1 = "Shreya"
s1.parents1()
```

Multilevel inheritance

- In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class.
- It is similar to a relationship involving grandparents and children.
- **Syntax:**

```
class classA:  
    <class-suite>  
class classB(classA):  
    <class suite>  
class classC(classB):  
    <class suite>
```



```
class Grandfather:
    def __init__(self, grandfathername):
        self.grandfathername = grandfathername
```

```
class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername

        # invoking constructor of Grandfather class
        Grandfather.__init__(self, grandfathername)
```

```
class Son(Father):
    def __init__(self, sonname, fathername, grandfathername):
        self.sonname = sonname

        Father.__init__(self, fathername, grandfathername)

    def print_name(self):
        print('Grandfather name :', self.grandfathername)
        print("Father name :", self.fathername)
        print("Son name :", self.sonname)
```

```
s1 = Son('Prince', 'Rampal', 'Lal mani')
print(s1.grandfathername)
s1.print_name()
```

Hierarchical Inheritance

- When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance.
- In this program, we have a parent (base) class and two or more child (derived) classes.

- **Syntax:**

```
class classA:
```

```
    <class-suite>
```

```
class classB(classA):
```

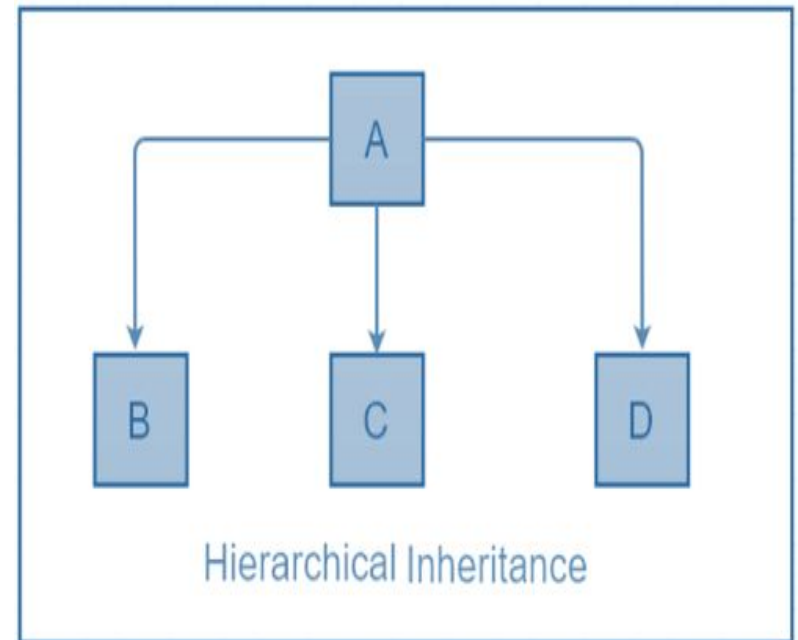
```
    <class suite>
```

```
class classC(classA):
```

```
    <class suite>
```

```
class classD(classA):
```

```
    <class suite>
```



```
class Parent:
    def func1(self):
        print("This function is in parent class.")
```

```
class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")
```

```
class Child2(Parent):
    def func3(self):
        print("This function is in child 2.")
```

```
object1 = Child1()
object2 = Child2()
object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

Hybrid Inheritance

- Hybrid Inheritance is a blend of more than one type of inheritance.
- Inheritance consisting of multiple types of inheritance is called hybrid inheritance.

```
class School:
    def func1(self):
        print("This function is in school.")

class Student1(School):
    def func2(self):
        print("This function is in student 1. ")

class Student2(School):
    def func3(self):
        print("This function is in student 2.")

class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")

# Driver's code
object = Student3()
object.func1()
object.func2()
```

super()

- `super()` provides the access to those methods of the super-class (parent class) which have been overridden in a sub-class (child class) that inherits from it.
- Python also has a `super()` function that will make the child class inherit all the methods and properties from its parent:
- By using the `super()` function, you do not have to use the name of the parent element, it will automatically inherit the methods and properties from its parent.

```
class Parent():  
    def show(self):  
        print("Inside Parent")  
class Child(Parent):  
    def show(self):  
        # Calling the parent's class method  
        super().show()  
        print("Inside Child")
```

```
obj = Child()  
obj.show()
```

Output:

```
Inside Parent  
Inside Child
```

```
class Person:  
    def __init__(self, fname, lname):  
        self.firstname = fname  
        self.lastname = lname  
  
    def printname(self):  
        print(self.firstname, self.lastname)
```

```
class Student(Person):  
    def __init__(self, fname, lname):  
        super().__init__(fname, lname)
```

```
x = Student("Mike", "Olsen")  
x.printname()
```

Add a property called graduation year to the Student class:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    def __init__(self, fname, lname):
        super().__init__(fname, lname)
        self.graduationyear = 2019

x = Student("Mike", "Olsen")
print(x.graduationyear)
```

In the example below, the year 2019 should be a variable, and passed into the Student class when creating student objects. To do so, add another parameter in the `__init__()` function:

Add a method called welcome to the Student class:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    def __init__(self, fname, lname, year):
        super().__init__(fname, lname)
        self.graduationyear = year

    def welcome(self):
        print("Welcome", self.firstname, self.lastname, "to the class of", self.graduationyear)

x = Student("Mike", "Olsen", 2024)
x.welcome()
```

If you add a method in the child class with the same name as a function in the parent class, the inheritance of the parent method will be overridden.

Benefits of inheritance are:

- It represents real-world relationships well.
- It provides the **reusability** of a code. We don't have to write the same code again and again. Also, it allows us to add more features to a class without modifying it.
- It is transitive in nature, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.
- Inheritance offers a simple, understandable model structure.
- Less development and maintenance expenses result from an inheritance.

is-a relationship

- Inheritance is an **is-a** relationship.
- inheritance is used only if there exists an **is-a** relationship between two classes.
- For example,
 - **Car** is a **Vehicle**
 - **Apple** is a **Fruit**
 - **Cat** is an **Animal**

isinstance()

- isinstance() method is used to check the relationship between the objects and classes. It returns true if the first parameter, i.e., obj is the instance of the second parameter, i.e., class.
- **Syntax:**
isinstance(Objectname, Classname)

Example:

```
class Calculation1:
```

```
    def Summation(self,a,b):
```

```
        return a+b;
```

```
class Calculation2:
```

```
    def Multiplication(self,a,b):
```

```
        return a*b;
```

```
class Derived(Calculation1,Calculation2):
```

```
    def Divide(self,a,b):
```

```
        return a/b;
```

```
d = Derived()
```

```
print(isinstance(d,Derived))
```

Polymorphism

- **Polymorphism** is a fundamental concept in object-oriented programming.
- Polymorphism means having many forms.
- Polymorphism means the same function name (but different signatures) being used for different types.
- that allows objects of different classes to be treated as objects of a common superclass. It enables the same interface to be used for different underlying forms (data types). In Python, polymorphism is typically achieved through method overriding and method overloading.
- **Method Overriding:**
- Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass. This is a form of polymorphism where a method behaves differently based on the object that is calling it.

method overloading

- Two or more methods have the same name but different numbers of parameters or different types of parameters, or both. These methods are called overloaded methods and this is called method overloading.

```
def product(a, b):
```

```
    p = a * b
```

```
    print(p)
```

```
def product(a, b, c):
```

```
    p = a * b*c
```

```
    print(p)
```

Method overriding

- *when a method in a subclass has the same name, parameters, and signature as a method in its parent class. By overriding a method, the subclass can provide a specific implementation of the method that is different from the one in its parent class.*
- Method overriding is an ability of any object-oriented programming language that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, the same parameters or signature, and same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to **override** the method in the super-class.

```
def add(x, y, z = 0):  
    return x + y+z
```

```
print(add(2, 3))
```

```
print(add(2, 3, 4))
```